Event Registration System

HackerRank | Java | Interview Problem + Solution

PART 1 — PROBLEM STATEMENT

Problem Statement

Implement an Event Registration System that manages different types of events, handles participant registrations, manages waitlists, and provides analytics. The system processes commands through standard input and outputs results to standard output.

Your task is to design and implement this system in Java.

Event Types

The system supports three types of events, each with different registration requirements.

1. CONFERENCE

- Requires business email addresses.
- The email domain must contain "company" OR "business".

Example valid emails:

```
john@company.com
jane@business.org
```

2. WORKSHOP

- Requires matching skill levels.
- Valid skill levels are: BEGINNER, INTERMEDIATE, ADVANCED
- A participant's skill level must match the required skill level of the workshop.

3. WEBINAR

- No special requirements.
- Anyone can register regardless of email or skill level.

Core Functionality

Your implementation must support the following features:

- Event creation with capacity limits and optional skill requirements.
- Participant registration with validation based on event type.
- Automatic waitlist management when an event reaches capacity.
- Waitlist promotion when a registration is cancelled.
- Duplicate registration prevention.
- Retrieval of event status and analytics.

Methods to Implement

Implement the following methods in the EventRegistrationSystem class.

Method	Returns / Notes
`addParticipant(id, name, email, skillLevel)`	Returns: Participant added: <id> Errors: Participant already exists, Invalid skill level
`createEvent(id, name, type, capacity, skillLevel)`	Returns: Event created: <id> Errors: Event already exists, Invalid event type, Invalid skill level
`registerParticipant( eventId, participantId)`	Returns: Registered: <p> for <e> OR Waitlisted: <p> for <e> Errors: Event not found, Participant not found, Already registered, Business email required, Skill level mismatch
`cancelRegistration( eventId, participantId)`	Returns: Cancelled: <p> OR Cancelled: <p>. Promoted: <next> OR Removed from waitlist: <p> Errors: Event not found, Participant not registered
`getEventStatus(eventId)`	Event: <id> | Registered: <n>/<cap> | Waitlist: <count>
`getAnalytics(eventId)`	Event: <id> | Fill rate: <X.X>% | Waitlist: <count>

Input Commands

The system reads commands from standard input. Supported commands:

```
ADD_PARTICIPANT <participantId> <name> <email> <skillLevel>

CREATE_EVENT <eventId> <name> <type> <capacity> [skillLevel]

REGISTER <eventId> <participantId>

CANCEL <eventId> <participantId>

STATUS <eventId>

ANALYTICS <eventId>
```

Example

Input

```
ADD_PARTICIPANT P1 John john@company.com BEGINNER
ADD_PARTICIPANT P2 Alice alice@gmail.com INTERMEDIATE
CREATE_EVENT E1 TechConf CONFERENCE 1
REGISTER E1 P1
REGISTER E1 P2
STATUS E1
ANALYTICS E1
```

Expected Output

```
Participant added: P1
Participant added: P2
Event created: E1
Registered: P1 for E1
Waitlisted: P2 for E1
Event: E1 | Registered: 1/1 | Waitlist: 1
Event: E1 | Fill rate: 100.0% | Waitlist: 1
```

PART 2 — SOLUTION

Design Overview

Key design decisions:

- List<String> + HashSet<String> for registered: maintains insertion order AND O(1) duplicate checks
- Queue<String> + HashSet<String> for waitlist: FIFO promotion AND O(1) membership checks
- Re-validates waitlisted participant on promotion (defensive, handles edge cases)
- Validation extracted into separate validate() method — clean, reusable
- Command parser in main() is clean switch/case — no if-else chains

> **Strategy Pattern note**
>
> The validate() method acts as a Strategy — dispatching logic per EventType.
> In an interview you can mention: 'I could extract this into a Validator interface with ConferenceValidator, WorkshopValidator, WebinarValidator — but for this problem size, the switch is clean and readable.'

Complete Java Solution

```
import java.util.*;

public class EventRegistrationSystem {

    enum EventType { CONFERENCE, WORKSHOP, WEBINAR }
    enum SkillLevel { BEGINNER, INTERMEDIATE, ADVANCED }

    static class Participant {
        String id, name, email;
        SkillLevel skillLevel;
        Participant(String id, String name, String email, SkillLevel
skillLevel) {
            this.id = id; this.name = name;
            this.email = email; this.skillLevel = skillLevel;
        }
    }

    static class Event {
        String id, name;
        EventType type;
        int capacity;
        SkillLevel requiredSkill;
        List<String> registered = new ArrayList<>();
        Queue<String> waitlist = new LinkedList<>();
        Set<String> registeredSet = new HashSet<>();
        Set<String> waitlistSet = new HashSet<>();

        Event(String id, String name, EventType type, int capacity,
SkillLevel req) {
            this.id = id; this.name = name; this.type = type;
            this.capacity = capacity; this.requiredSkill = req;
        }
        boolean isFull() { return registered.size() >= capacity; }
    }

    Map<String, Event> events = new HashMap<>();
    Map<String, Participant> participants = new HashMap<>();

    // 1. Add Participant
    String addParticipant(String id, String name, String email, String
skill) {
        if (participants.containsKey(id)) return "ERROR: Participant
already exists";
        SkillLevel skillLevel = null;
        if (skill != null && !skill.isEmpty()) {
            try { skillLevel = SkillLevel.valueOf(skill); }
            catch (Exception e) { return "ERROR: Invalid skill level"; }
        }
        participants.put(id, new Participant(id, name, email,
skillLevel));
        return "Participant added: " + id;
    }

    // 2. Create Event
    String createEvent(String id, String name, String type, int capacity,
String skill) {
```

```
        if (events.containsKey(id)) return "ERROR: Event already exists";
        EventType eventType;
        try { eventType = EventType.valueOf(type); }
        catch (Exception e) { return "ERROR: Invalid event type"; }
        SkillLevel skillLevel = null;
        if (eventType == EventType.WORKSHOP) {
            try { skillLevel = SkillLevel.valueOf(skill); }
            catch (Exception e) { return "ERROR: Invalid skill level"; }
        }
        events.put(id, new Event(id, name, eventType, capacity,
skillLevel));
        return "Event created: " + id;
    }

    // 3. Register Participant
    String registerParticipant(String eventId, String participantId) {
        Event event = events.get(eventId);
        if (event == null) return "ERROR: Event not found";
        Participant p = participants.get(participantId);
        if (p == null) return "ERROR: Participant not found";
        if (event.registeredSet.contains(participantId) ||
            event.waitlistSet.contains(participantId))
            return "ERROR: Already registered";
        String err = validate(event, p);
        if (err != null) return err;
        if (!event.isFull()) {
            event.registered.add(participantId);
            event.registeredSet.add(participantId);
            return "Registered: " + participantId + " for " + eventId;
        } else {
            event.waitlist.add(participantId);
            event.waitlistSet.add(participantId);
            return "Waitlisted: " + participantId + " for " + eventId;
        }
    }

    // 4. Cancel Registration
    String cancelRegistration(String eventId, String participantId) {
        Event event = events.get(eventId);
        if (event == null) return "ERROR: Event not found";
        if (event.registeredSet.contains(participantId)) {
            event.registered.remove(participantId);
            event.registeredSet.remove(participantId);
            // Promote from waitlist
            while (!event.waitlist.isEmpty()) {
                String nextId = event.waitlist.poll();
                event.waitlistSet.remove(nextId);
                Participant next = participants.get(nextId);
                if (next != null && validate(event, next) == null) {
                    event.registered.add(nextId);
                    event.registeredSet.add(nextId);
                    return "Cancelled: " + participantId + ". Promoted: "
+ nextId;
                }
            }
            return "Cancelled: " + participantId;
        } else if (event.waitlistSet.contains(participantId)) {
            event.waitlist.remove(participantId);
            event.waitlistSet.remove(participantId);
```

```
            return "Removed from waitlist: " + participantId;
        }
        return "ERROR: Participant not registered";
    }

    // 5. Get Event Status
    String getEventStatus(String eventId) {
        Event event = events.get(eventId);
        if (event == null) return "ERROR: Event not found";
        return String.format("Event: %s | Registered: %d/%d | Waitlist:
%d",
            event.id, event.registered.size(), event.capacity,
event.waitlist.size());
    }

    // 6. Get Analytics
    String getAnalytics(String eventId) {
        Event event = events.get(eventId);
        if (event == null) return "ERROR: Event not found";
        double fillRate = (double) event.registered.size() /
event.capacity * 100;
        return String.format("Event: %s | Fill rate: %.1f%% | Waitlist:
%d",
            event.id, fillRate, event.waitlist.size());
    }

    // ─── Validation ──────────────────────────────────────────
    private String validate(Event event, Participant p) {
        switch (event.type) {
            case CONFERENCE:
                if (!isBusinessEmail(p.email))
                    return "ERROR: Business email required for
CONFERENCE";
                break;
            case WORKSHOP:
                if (p.skillLevel == null || p.skillLevel !=
event.requiredSkill)
                    return "ERROR: Skill level mismatch for WORKSHOP";
                break;
            case WEBINAR: break; // no restrictions
        }
        return null;
    }

    private boolean isBusinessEmail(String email) {
        if (email == null) return false;
        String domain = email.contains("@") ?
email.substring(email.indexOf("@") + 1) : "";
        return domain.contains("company") || domain.contains("business");
    }

    // ─── Main / Command Parser ───────────────────────────────
    public static void main(String[] args) {
        EventRegistrationSystem sys = new EventRegistrationSystem();
        Scanner scanner = new Scanner(System.in);
        while (scanner.hasNextLine()) {
            String line = scanner.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
```

```
            String result;
            switch (p[0].toUpperCase()) {
                case "ADD_PARTICIPANT":
                    result = sys.addParticipant(p[1], p[2], p[3],
                        p.length > 4 ? p[4] : null); break;
                case "CREATE_EVENT":
                    result = sys.createEvent(p[1], p[2], p[3],
                        Integer.parseInt(p[4]),
                        p.length > 5 ? p[5] : null); break;
                case "REGISTER":
                    result = sys.registerParticipant(p[1], p[2]); break;
                case "CANCEL":
                    result = sys.cancelRegistration(p[1], p[2]); break;
                case "STATUS":
                    result = sys.getEventStatus(p[1]); break;
                case "ANALYTICS":
                    result = sys.getAnalytics(p[1]); break;
                default:
                    result = "ERROR: Unknown command";
            }
            System.out.println(result);
        }
    }
}
```

Solution Walkthrough

Data Structures Used

Field	Type	Why
`registered`	`List<String>`	Maintains insertion order
`registeredSet`	`HashSet<String>`	O(1) duplicate check
`waitlist`	`Queue<String>`	FIFO — fair ordering
`waitlistSet`	`HashSet<String>`	O(1) membership check

Edge Cases Handled

- Duplicate registration check covers BOTH registered and waitlisted sets
- WORKSHOP validation requires both non-null and exact match on skillLevel
- Promotion loop re-validates — skips ineligible waitlisted participants
- isBusinessEmail() extracts domain after '@' safely
- createEvent() only requires skillLevel for WORKSHOP type
- addParticipant() allows null skill (needed for WEBINAR/CONFERENCE participants)

Interview tip — what to say out loud

1. 'I'm using two data structures per collection — a List/Queue for ordering, plus a Set for O(1) duplicate checking.'

2. 'I separated validation into its own method to keep registerParticipant clean.'
3. 'The promotion loop re-validates in case business rules changed — defensive coding.'
4. 'I used enums for EventType and SkillLevel to catch invalid inputs at parse time.'